



UNIVERSITY
OF TRIESTE

Anti-Jamming Communications Using a Deep Reinforcement Learning Approach

September 2026

Reinforcement Learning · Course Project · Università degli Studi di Trieste

Tavano Matteo · SM3800057

Project Outline

- | | | |
|-----------|----------------------------------|---|
| 01 | Introduction | The jamming problem |
| 02 | Problem Formulation | Objective, environment, jamming patterns |
| 03 | MDP Formulation | State, action, reward, observability |
| 04 | Methods | Baselines, Q-learning, DQN |
| 05 | Optimal Reference | r^* via policy iteration |
| 06 | Experiments & Results | Experimental Setup & results presentation |
| 07 | Conclusions | Conclusions & discussion |

01 Introduction

The anti-jamming problem

A transmitter must choose one of **K frequency channels** at every time step. A jammer occupies a subset of those channels, aiming to block the transmission.

Collision

Transmitting on a jammed channel is ineffective.

Switching cost

Changing frequency costs time and delays communication.

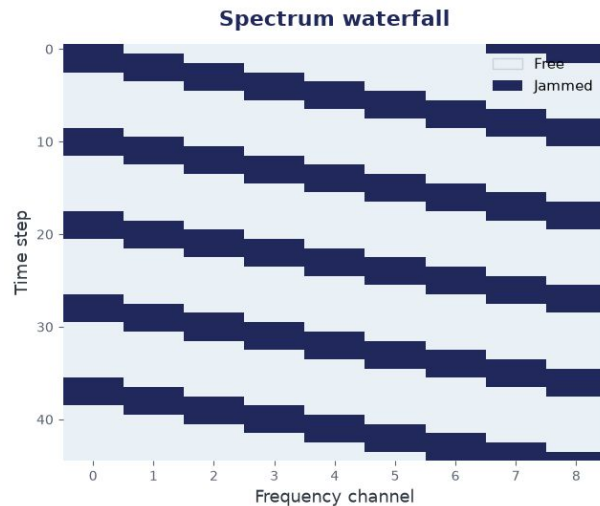
Unknown pattern

The agent is not informed of the jammer's pattern.

Goal: The agent aims to maintain **reliable communication** by avoiding jammed channels while minimizing unnecessary channel switching.

Learning to Avoid a Jammer

- The **Deep RL** agent learns to predict and exploit available channels from the observed spectrum, without prior knowledge of the jammer's dynamics.
- It uses a history of past spectrum observations to adapt its channel-selection policy and maintain reliable communication while limiting unnecessary channel switching.



02 Problem Formulation

Objective

The agent **maximizes** the long-run **average reward per step**. The task is **continuing**, with no episode boundary.

$$\max_{\pi} \lim_{N \rightarrow \infty} \frac{1}{N} \mathbb{E}_{\pi} \left[\sum_{t=1}^N r_t \right]$$

$$r_t = \mu_t - \lambda \delta(a_t \neq a_{t-1})$$

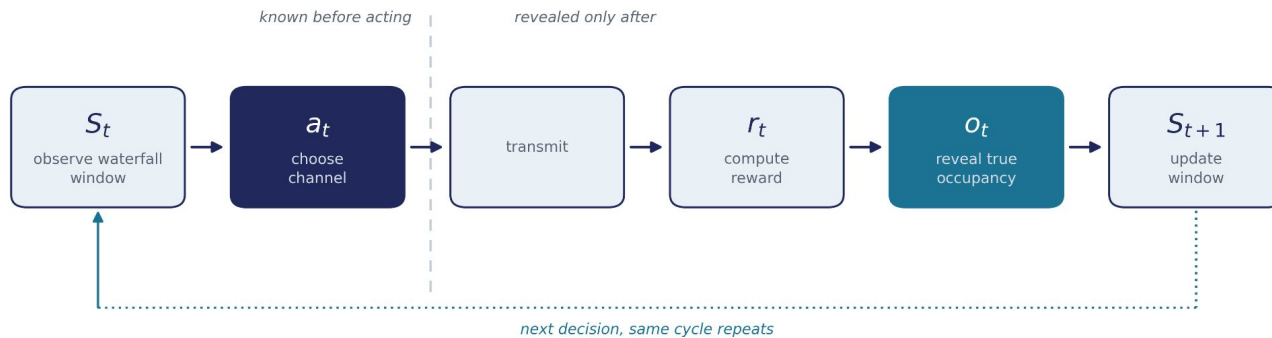
$$\mu_t \in \{0, 1\} \quad (\text{transmission success}) \quad \lambda = 0.2 \quad (\text{switching cost})$$

Reward trades transmission success against the cost of changing channel.

Q-learning and DQN optimize a discounted return ($\gamma = 0.99$) as a proxy for the average reward ($\gamma \rightarrow 1$).

Environment Dynamics

Environment timing: sense-then-transmit



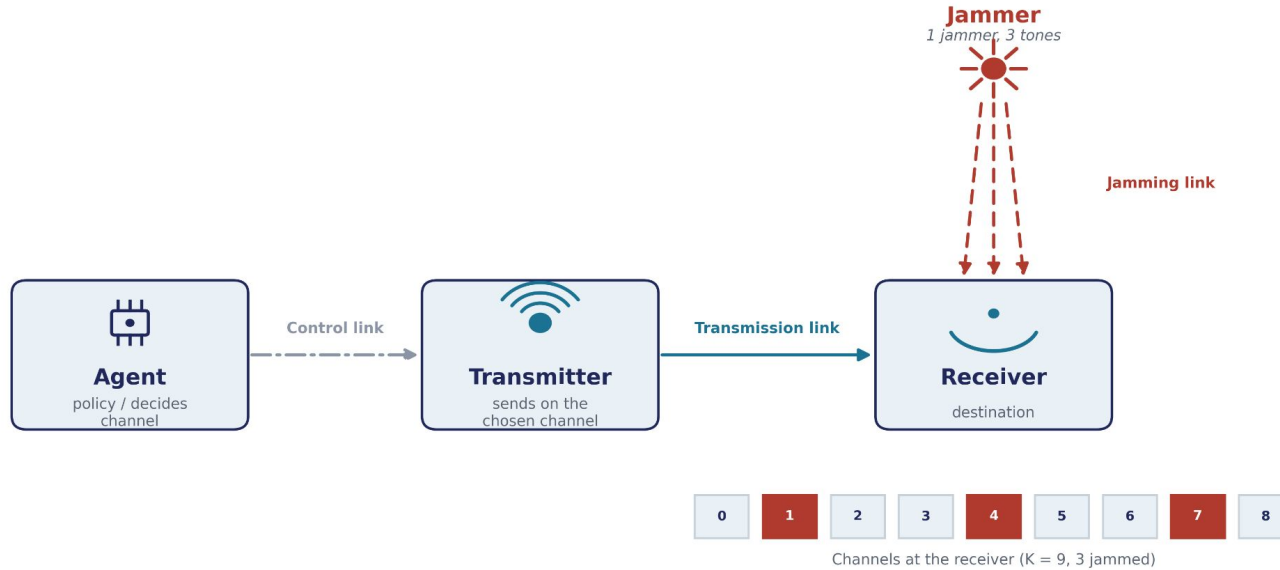
S_t contains the full-spectrum occupancy up to $t-1$. The occupancy o_t of the current step is revealed only after transmitting.

Anti-jamming network topology

— Transmission link

- - - Control link

- - - Jamming link



Three jamming patterns

Sweep

m contiguous tones advance by one slot each step, cyclically.

Deterministic dynamics,
random initialization

Dynamic comb

m tones stay fixed for $D = 10$ steps, then are redrawn uniformly at random
(m distinct channels)

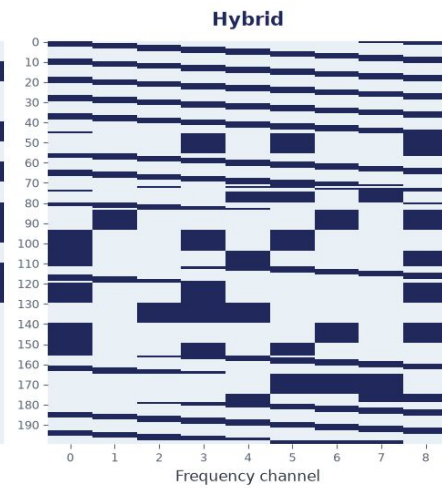
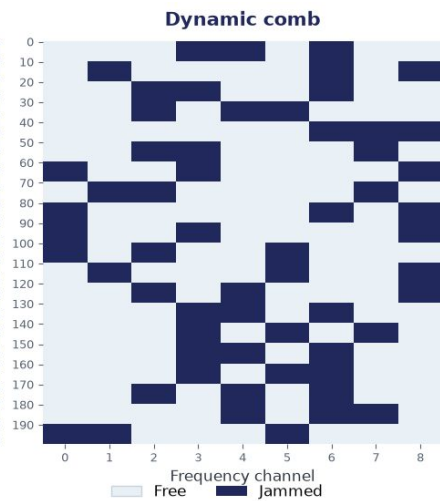
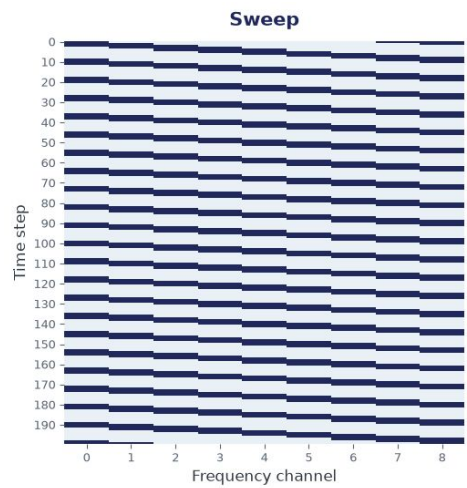
Stochastic resamples after
D steps

Hybrid

At each step the regime flips (sweep $\leftrightarrow$ comb) with probability $p = 0.02$

Stochastic change of
regimes

All three are exogenous: the jammer never reacts to the agent's choice.



03 MDP Formulation

State, action, reward

State	Last $T = 20$ full-spectrum readings, (DQN).
Action	One of $K = 9$ channels, $a_t \in \{0, \dots, K-1\}$
Reward	Transmission success, considering the switching cost. It depends on o_t (not yet observed) and on a_{t-1} .
Transition	The jammer evolves on its own; S_{t+1} is S_t shifted, with o_t appended

$$S_t = (o_{t-T}, o_{t-T+1}, \dots, o_{t-1}) \in \{0, 1\}^{T \times K}$$

$K = 9$ channels, $m = 3$ jammed tones, $T = 20$ observation memory.

Observability: a POMDP handled with a history window

Hidden jammer state z_t

Markov by construction: the next configuration depends only on the current one.

$$P(z_{t+1} \mid z_t, a_t) = P(z_{t+1} \mid z_t)$$

What the agent sees

The agent sees $\mathbf{o}_{t-1} = \mathbf{f}(z_{t-1})$: the exact occupancy, but one step old and blind to hidden variables (dwell counter, regime)

$\mathbf{o}_{t-1} = \mathbf{f}(z_{t-1})$; $\mathbf{f}(z_t)$ is revealed only after acting



Methods

Methods compared

Baselines

- **Fixed:** always the same channel
- **Random:** uniform choice
- **Reactive:** stay if the channel was free at $t-1$, else jump to a random channel

Learning Algorithms

- **Q-learning:** tabular, reduced state.
- **DQN:** CNN over the full waterfall (TxK)

Reactive asks the real question: is learning worth it here?

Baselines do not learn: they are fixed decision rules used as reference

Tabular Q-learning

Acts on a **reduced state**: the set of channels observed as jammed at $t-1$, and the previous action

$$\phi_t = (\{k : o_{t-1}[k] = 1\}, a_{t-1})$$

Temporal-difference (TD) update with **ϵ -greedy** exploration:

$$Q(\phi_t, a_t) \leftarrow Q(\phi_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(\phi_{t+1}, a') - Q(\phi_t, a_t)]$$

Parameters: $\alpha = 0.1$ $\gamma = 0.99$ ϵ decays linearly: $1.0 \rightarrow 0.05$ over 2 500 steps

No temporal memory beyond a single step. Tabular Q-learning also suffers from the curse of dimensionality.

Algorithm 1 Tabular Q-learning with reduced state

```
Input:  $K, \alpha, \gamma, \epsilon_{\text{start}}, \epsilon_{\text{end}}, \epsilon_{\text{decay}}$ 
Initialize:  $Q(\phi, a) \leftarrow 0$  for all  $\phi, a$  (lazily, on first visit)
Initialize:  $a_{\text{last}} \leftarrow -1, \text{step} \leftarrow 0$ 

for each environment step  $t = 0, 1, 2, \dots$ 
    observe  $S_t$   $\triangleright$  waterfall window, shape  $(T, K)$ 
     $\phi_t \leftarrow (\text{jammed\_channels}(S_t[-1]), a_{\text{last}})$   $\triangleright$  reduce: last row + previous action

     $\epsilon \leftarrow \epsilon_{\text{start}} + \min(1, \text{step} / \epsilon_{\text{decay}}) \cdot (\epsilon_{\text{end}} - \epsilon_{\text{start}})$ 
    with probability  $\epsilon$ :
         $a_t \leftarrow \text{random action in } \{0, \dots, K-1\}$   $\triangleright$  explore
    else:
         $a_t \leftarrow \text{argmax}_a Q(\phi_t, a)$   $\triangleright$  exploit – ties broken uniformly at random

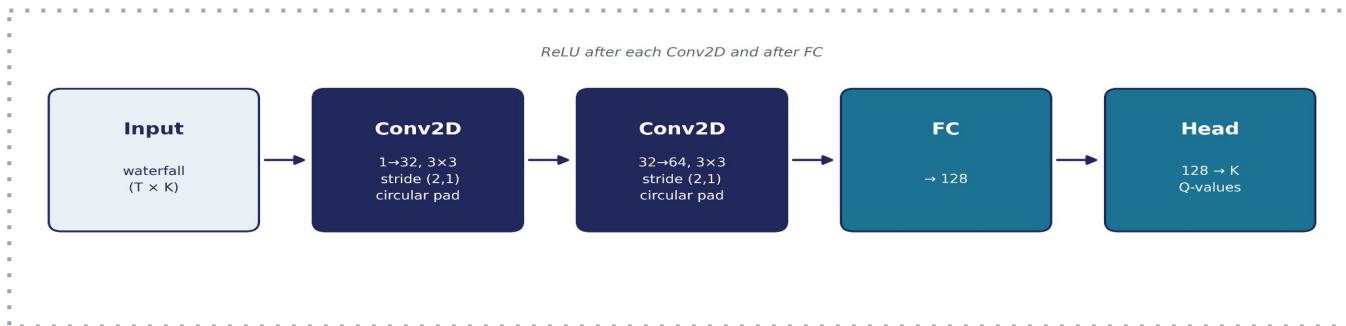
    execute  $a_t$ , observe  $r_t, S_{\{t+1\}}$ 

     $\phi_{\{t+1\}} \leftarrow (\text{jammed\_channels}(S_{\{t+1\}}[-1]), a_t)$   $\triangleright$  the action just taken becomes “last action”
     $\text{target} \leftarrow r_t + \gamma \cdot \max_a Q(\phi_{\{t+1\}}, a)$ 
     $Q(\phi_t, a_t) \leftarrow Q(\phi_t, a_t) + \alpha \cdot (\text{target} - Q(\phi_t, a_t))$   $\triangleright$  TD update

     $a_{\text{last}} \leftarrow a_t, \text{step} \leftarrow \text{step} + 1$ 
end for
```

Deep Q-Network

- DQN replaces the lookup table with a neural network that estimates $Q(s, a)$ directly from the raw waterfall.
- Two things keep learning stable: a **replay buffer D** , which samples random past transitions, breaking temporal correlation, and a **target network θ'** , a slightly older copy used only to keep the learning target from shifting



Algorithm 2 Deep Q-Network (DQN)

```
Input:  $T, K, \gamma, \text{learning\_rate}, \epsilon_{\text{start}}, \epsilon_{\text{end}}, \epsilon_{\text{decay}}$ 
Input:  $\text{replay\_capacity}, \text{warmup\_size}, \text{batch\_size}, \text{target\_sync\_steps}$ 
Initialize: online network  $Q_{\theta}$ , target network  $Q_{\theta^-} \leftarrow Q_{\theta}$ 
Initialize: replay buffer  $D$  (capacity =  $\text{replay\_capacity}$ ),  $\text{step} \leftarrow 0$ 

for each environment step  $t = 0, 1, 2, \dots$ 
    observe  $S_t$  ▷ waterfall window, shape  $(T, K)$ 

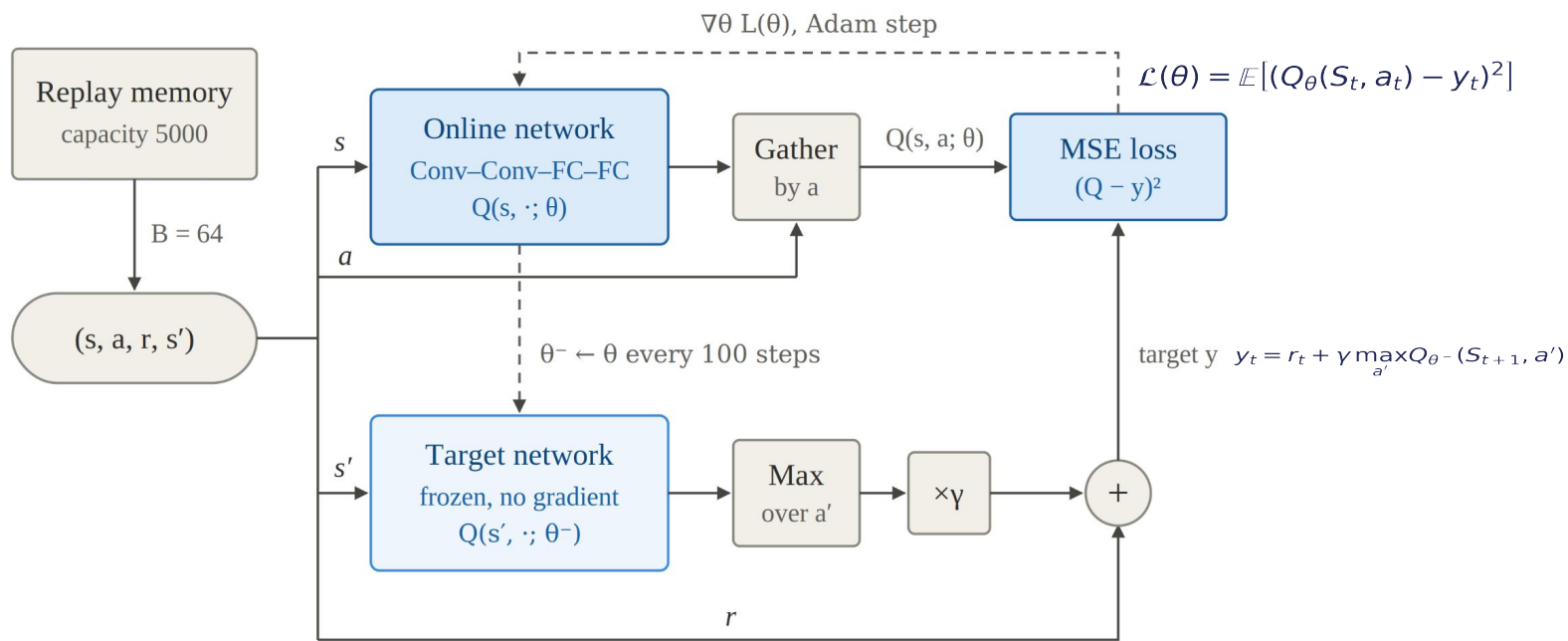
     $\epsilon \leftarrow \epsilon_{\text{start}} + \min(1, \text{step} / \epsilon_{\text{decay}}) \cdot (\epsilon_{\text{end}} - \epsilon_{\text{start}})$ 
    with probability  $\epsilon$ :
         $a_t \leftarrow \text{random action in } \{0, \dots, K-1\}$  ▷ explore
    else:
         $a_t \leftarrow \text{argmax}_a Q_{\theta}(S_t, a)$  ▷ exploit – no gradient tracked

    execute  $a_t$ , observe  $r_t, S_{t+1}$ 
    store  $(S_t, a_t, r_t, S_{t+1})$  in  $D$  ▷ ring buffer, oldest dropped once full
     $\text{step} \leftarrow \text{step} + 1$ 

    if  $|D| \geq \max(\text{warmup\_size}, \text{batch\_size})$ :
        sample batch  $\{(s_i, a_i, r_i, s_{i'})\}_{i=1.. \text{batch\_size}}$  from  $D$  ▷ uniform, i.i.d.
         $y_i \leftarrow r_i + \gamma \cdot \max_a Q_{\theta^-}(s_{i'}, a)$  ▷ target network, no gradient
         $\text{loss} \leftarrow (1/\text{batch\_size}) \cdot \sum_i (Q_{\theta}(s_i, a_i) - y_i)^2$ 
         $\theta \leftarrow \theta - \text{Adam\_update}(\nabla_{\theta} \text{loss}, \text{learning\_rate})$ 

    if  $\text{step} \bmod \text{target\_sync\_steps} = 0$ :
         $Q_{\theta^-} \leftarrow Q_{\theta}$  ▷ sync target network

end for
```



05 Optimal Long-term average reward

Why an optimal reference

A raw reward has no absolute meaning: r^* gives the best attainable value and turns it into a percentage.

r^* is the **optimal average reward** of the fully-observed MDP.

it upper-bounds any policy that only observes the waterfall.

η says how much of the achievable performance a policy actually captures.

$$\eta = 100 \cdot \frac{\bar{r}}{r^*} \% \leq 100 \%$$

Algorithm 3 Policy Iteration (average reward)

```
Input:  $S, A, P(s'|s,a), r(s,a), \text{max\_iter}, \epsilon, \text{stable\_iters}, \text{seed}$ 
Initialize:  $s\_ref \leftarrow S[0], \text{ gauge } h(s\_ref) = 0, \pi(s) \leftarrow \text{random action } \forall s \in S$ 
Initialize:  $\text{prev\_p} \leftarrow \text{None}, \text{ stable} \leftarrow 0$ 

for  $k = 1, 2, \dots, \text{max\_iter}$ :
    solve for  $(p, h)$ :  $\triangleright$  policy evaluation,  $h(s\_ref) = 0$ 
         $p + h(s) - \sum_{s'} P(s'|s,\pi(s)) \cdot h(s') = r(s,\pi(s)) \quad \forall s \in S$ 
    if system singular:  $\triangleright$  sparse solve fails or non-finite
         $\pi(s) \leftarrow \text{random action } \forall s; \text{ prev\_p} \leftarrow \text{None}; \text{ stable} \leftarrow 0; \text{ next } k$ 

    if  $\text{prev\_p} \neq \text{None}$  and  $|p - \text{prev\_p}| < \epsilon$ :  $\text{stable} \leftarrow \text{stable} + 1$ 
    else:  $\text{stable} \leftarrow 0$ 
     $\text{prev\_p} \leftarrow p$ 
    if  $\text{stable} \geq \text{stable\_iters}$ : return  $p, h, \pi$   $\triangleright$  stops on  $p$ , not on  $\pi$ 

    for each  $s \in S$ :  $\triangleright$  policy improvement
         $\pi(s) \leftarrow \text{argmax}_a [r(s,a) + \sum_{s'} P(s'|s,a) \cdot h(s')] \quad \triangleright$  greedy w.r.t.  $h$ , ties  $\rightarrow$  first  $a$ 
    end for
return  $p, h, \pi$   $\triangleright$  max_iter reached
```

Closed-form derivations

Two of the three scenarios admit a closed form, which cross-checks the numerical result.

sweep $r_{\text{sweep}}^* = 1 - \frac{\lambda}{K-m} = 0.9667$

Stay $K-m$ steps on a free channel, then switch once.

dynamic comb $r_{\text{dyn comb}}^* = \frac{\mathbb{E}[\text{reward per cycle}]}{D} = \frac{(1-m/K)D + (m/K)(D-1-\lambda)}{D} = 0.9600$

Per reshuffle cycle: safe with prob. $1-m/K$, else one lost step plus a switch.

hybrid $r_{\text{hybrid}}^* = 0.9616$ (no closed form)

Mixed dynamics: computed using policy iteration only.

Scenario	r^*
Sweep	0.9667
Dynamic comb	0.9600
Hybrid	0.9616



Experiments & Results

Setup and hyperparameters

Environment

$K = 9$ channels · $m = 3$ tones · $\lambda = 0.2$ · $T = 20$

Scenarios

sweep · dynamic comb (Dwell = 10) · hybrid ($p = 0.02$, per-step regime-switch probability)

Q-learning

$\alpha = 0.1$ · $\gamma = 0.99$ · ϵ 1.0 $\rightarrow$ 0.05

DQN

$lr = 5 \cdot 10^{-4}$ · batch 64 · replay 5 000 · target sync 100

Protocol

10 000 training steps · 2 000 evaluation steps · 5 training seeds · 10 test seeds

Experimental Settings

In-domain

Frozen policy, same seed used for training

Did it learn the training instance?

Cross-seed

Same jamming pattern, unseen seed

A general rule, or a memorised instance?

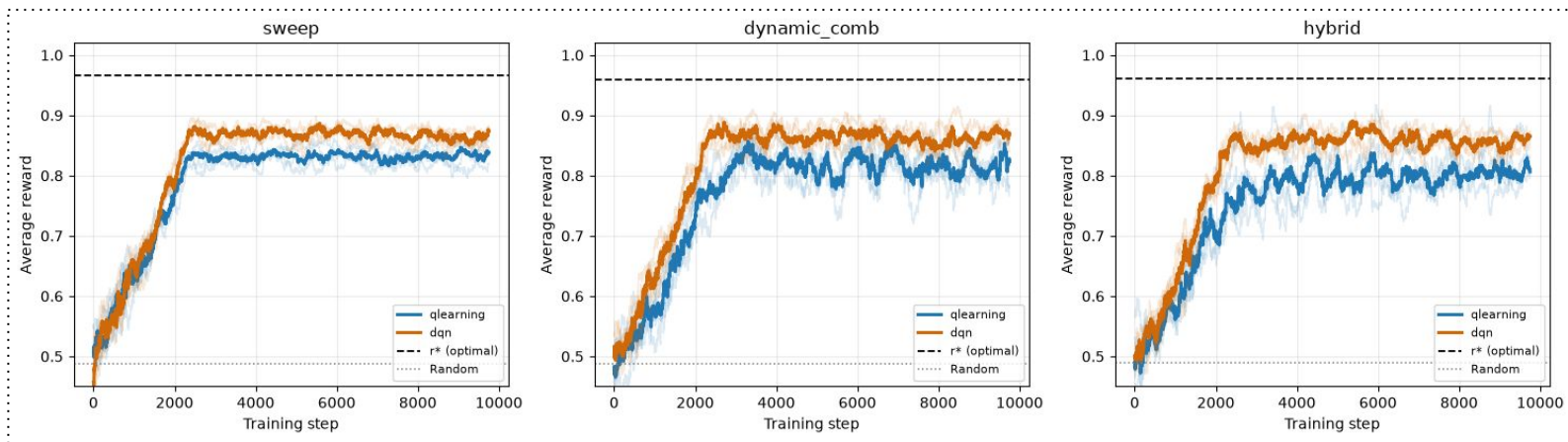
Cross-domain

A different jamming pattern

Does the strategy transfer?

Evaluation always freezes the policy and removes exploration.

Training curves



Both algorithms plateau after $\approx 2,500$ steps, below r^ .
Training reward includes $\epsilon = 0.05$ exploration.*

In-domain performance

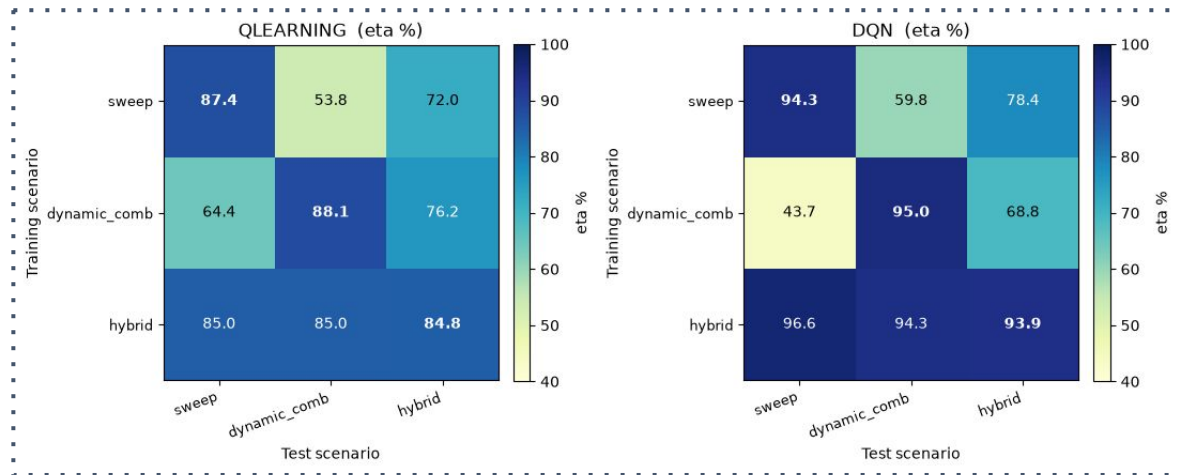
$\bar{r}$ (reward) $\bar{\mu}$ (throughput) $\bar{\sigma}$ (switching rate)

- Reward alone hides the cause of a gap: separates **collisions** from unnecessary **switching**.
- Averaged over evaluation steps and over same training seeds; median across the five training runs.
- Training curves are smoothed over a **moving window**, and still include residual exploration.

Decomposing the reward shows whether a gap comes from collisions or from switching.

Sweep					
Method	$\bar{r}$	$\bar{\mu}$	$\bar{\sigma}$	η (%)	Train time (s)
Q-learning	0.8446	1.0000	0.7770	87.37	0.3
DQN	0.9112	1.0000	0.4440	94.26	42.9
Fixed	0.6670	0.6670	0.0000	69.00	–
Random	0.4869	0.6645	0.8895	50.37	–
Reactive	0.6556	0.7130	0.2870	67.82	–
Dynamic comb					
Method	$\bar{r}$	$\bar{\mu}$	$\bar{\sigma}$	η (%)	Train time (s)
Q-learning	0.8515	0.9625	0.5250	88.70	0.3
DQN	0.9194	0.9655	0.2670	95.77	37.2
Fixed	0.6450	0.6450	0.0000	67.19	–
Random	0.4876	0.6655	0.8895	50.79	–
Reactive	0.9502	0.9585	0.0415	98.98	–
Hybrid					
Method	$\bar{r}$	$\bar{\mu}$	$\bar{\sigma}$	η (%)	Train time (s)
Q-learning	0.8219	0.9665	0.7160	85.48	0.9
DQN	0.9066	0.9805	0.3940	94.28	35.6
Fixed	0.6585	0.6585	0.0000	68.48	–
Random	0.4896	0.6665	0.8895	50.92	–
Reactive	0.8093	0.8410	0.1585	84.17	–

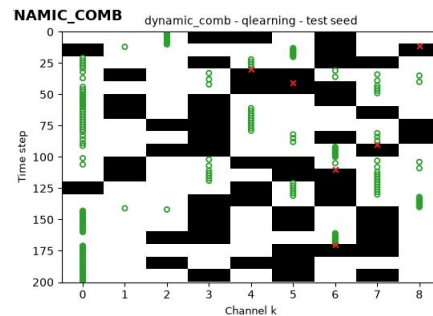
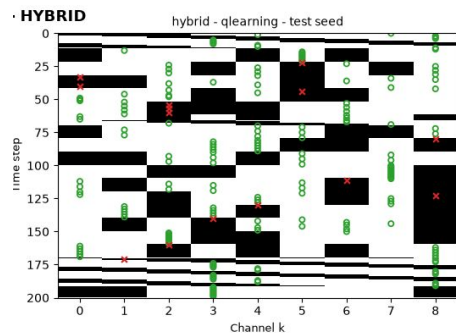
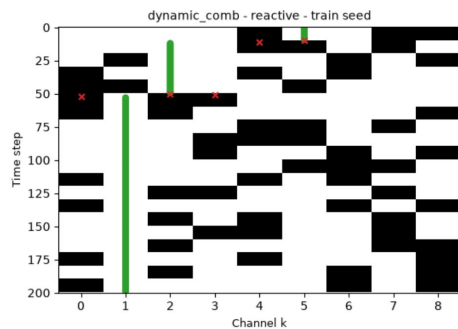
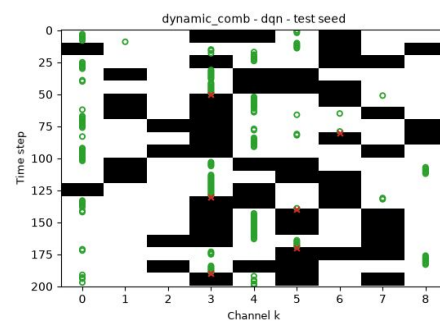
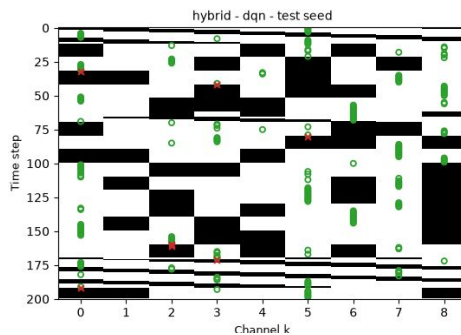
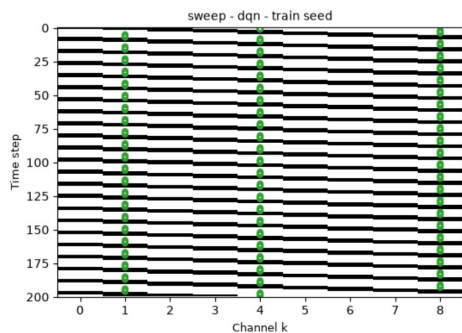
Cross-seed and Cross-domain



*Diagonal values are cross-seed results: on unseen seeds η stays close to the in-domain values.
Training on hybrid also transfers well to the other two scenarios*

Results

Learned behaviour





Conclusions

Key takeaways

Both algorithms converge within ~2,500 steps to policies that outperform Random and Fixed, reaching 85–89% (Q-learning) and 94–96% (DQN) of the theoretical optimum r^ .*

A reproducible simulation framework with three jamming patterns and two learning algorithms.

*An **optimal long-term average reward** was defined and validated, giving every method a common, exact benchmark. r^* is an upper bound from the fully observed MDP.*

***Cross-seed transfer** is strong for both algorithms, the policies learn a general rule, not one memorized instance.*

***Cross-domain transfer** matches expectations: it fails where the training source is not informative enough.*

Limitations and Future Work

Limitations

The jamming pattern is static and *non-adaptive*.

The experimental setup is a simplification of a real system.

In dynamic comb, RL cannot beat the simple Reactive rule.

Future Work

Improve the architecture and explore new algorithms.

Study multichannel communication and scale up the problem.

Explore new *jamming patterns* and increasing number of tones.

Thanks for listening